

3. Header Linked List

–Insert At Beginning.

Program:

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    // Link new node to the current first node
    newNode->next = head->next;

    // Link header to the new node
    head->next = newNode;

    // Increment internal count
    head->data++;

    cout << data << " inserted at the beginning." << endl;
}

...
case 1:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtBeginning(data);
    break;
```

Program Analysis

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a *Node* structure and *newNode* is declared as a pointer to *Node* structure.

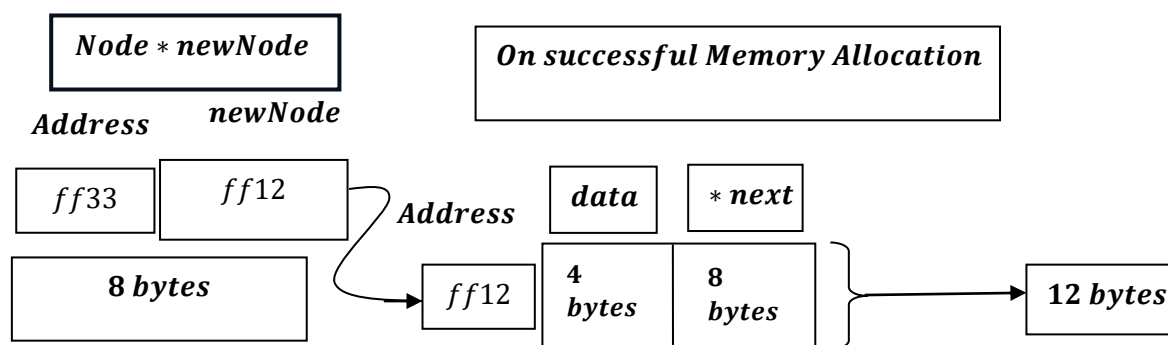
→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the *Node* structure itself is determined by the sum of the sizes of its members:

→ *data*(an integer) is typically 4 bytes.

→ *next*(a pointer to another *Node*) is typically 8 bytes.

→ Therefore ,the size of the *Node* structure is typically 12 bytes(4 bytes for *data* + 8 bytes for *next*) .



```

if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}

```

Now newNode = ff12, hence → Memory Allocation is Successful !

if newNode = NULL , then :

Message → "Error: Memory Allocation for Header Failed."

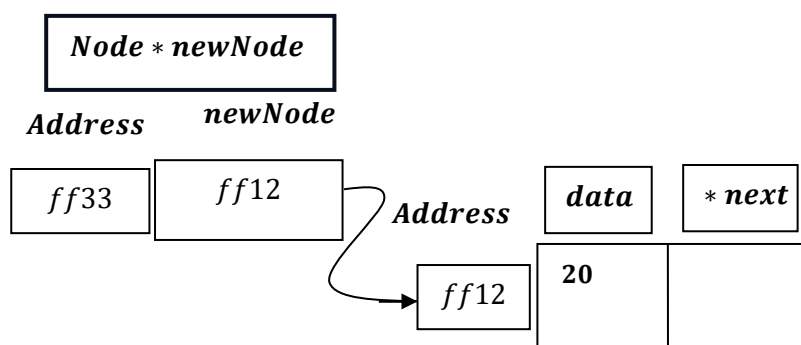
And then reurn will return void and exit from the module (function).

```

newNode->data = data;

```

Lets say , data = 20.

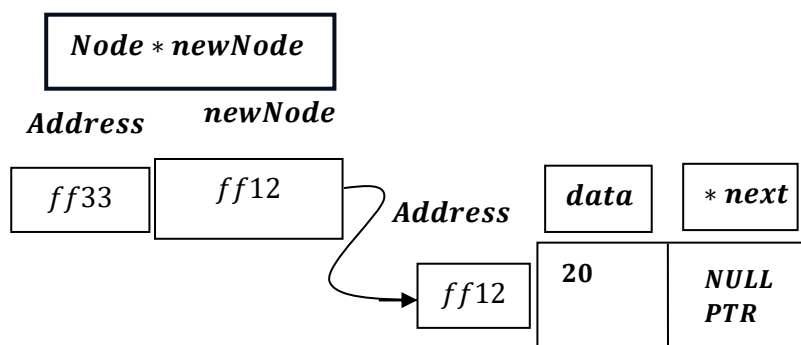
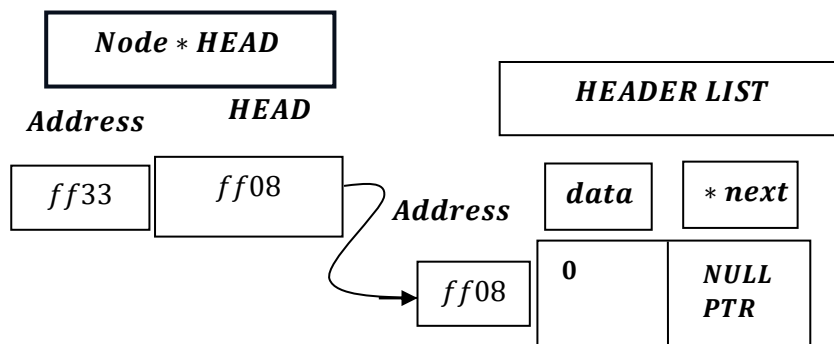


```

newNode->next = head->next;

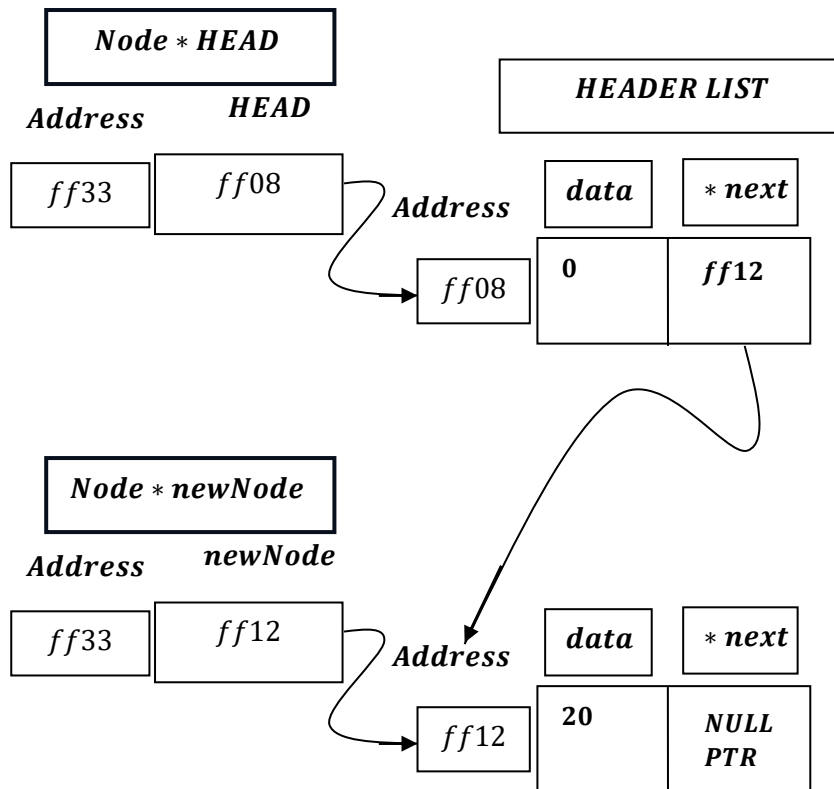
```

newNode → next = head → next = NULLPTR, as we get from:



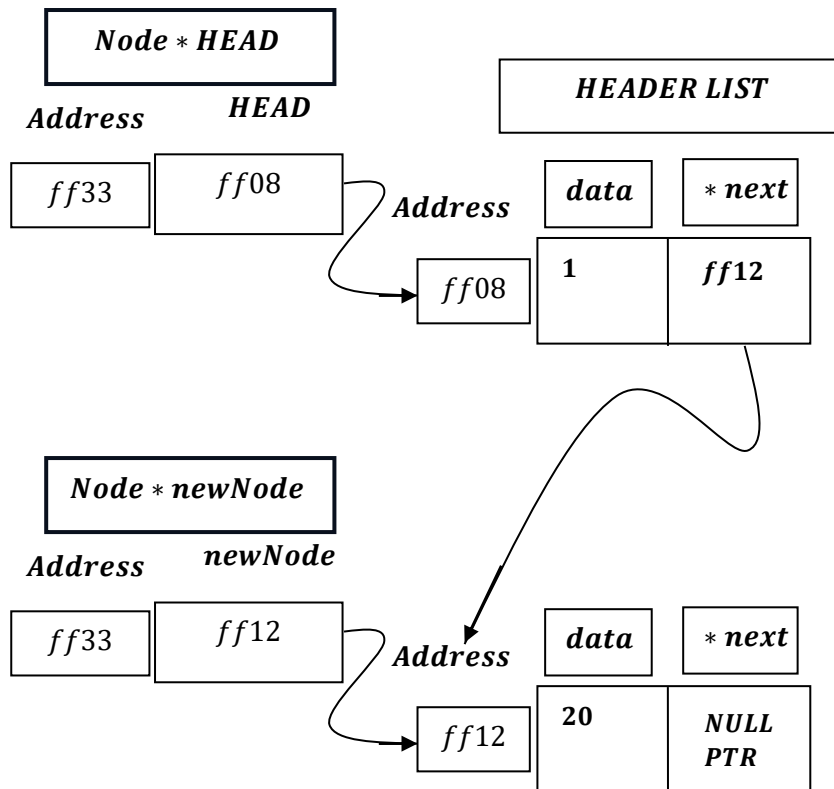
```
head->next = newNode;
```

head $\rightarrow$ ***next*** = ***newNode*** = ***ff12***.

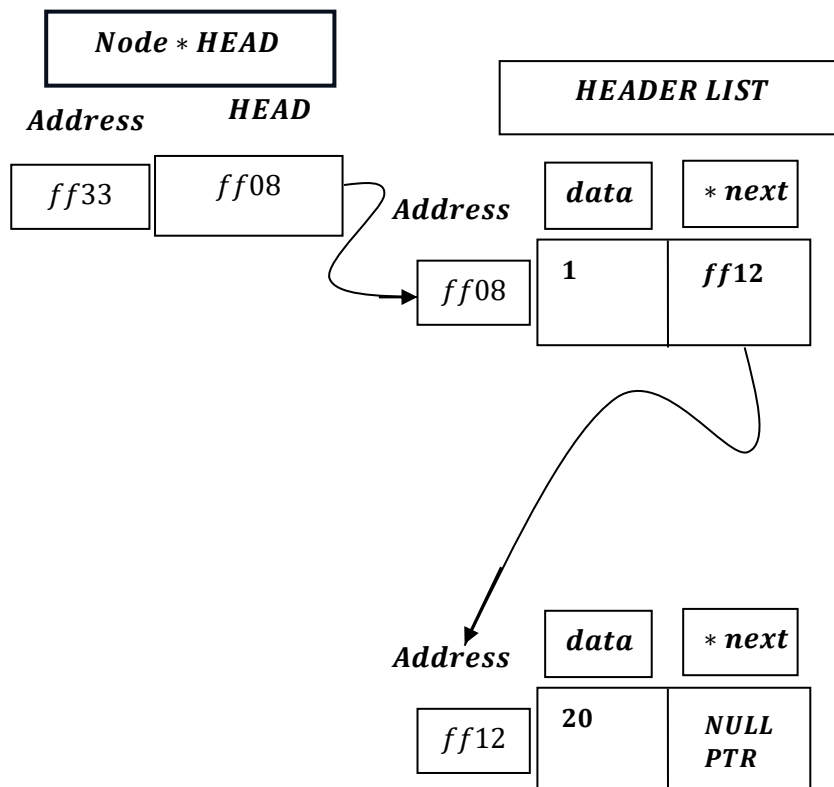


```
head->data++;
```

$Head \rightarrow data ++ \Rightarrow Head \rightarrow data \Rightarrow Head \rightarrow data + 1 \Rightarrow 0 + 1 = 1.$

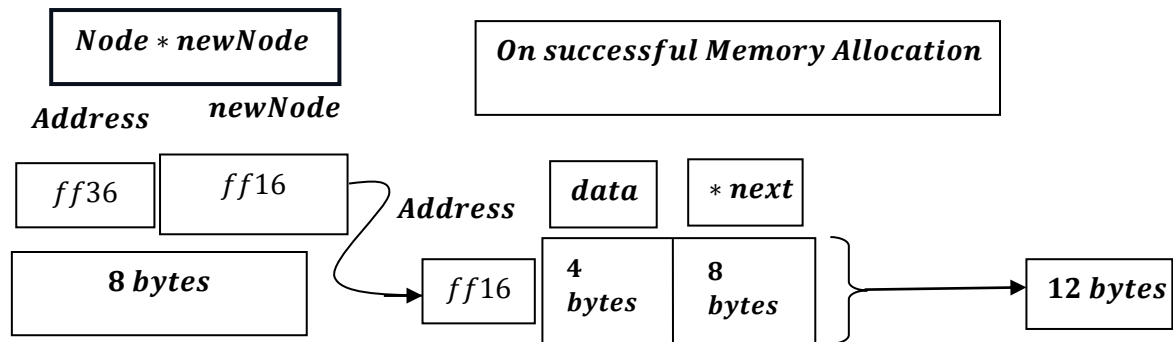


And we got our first list:



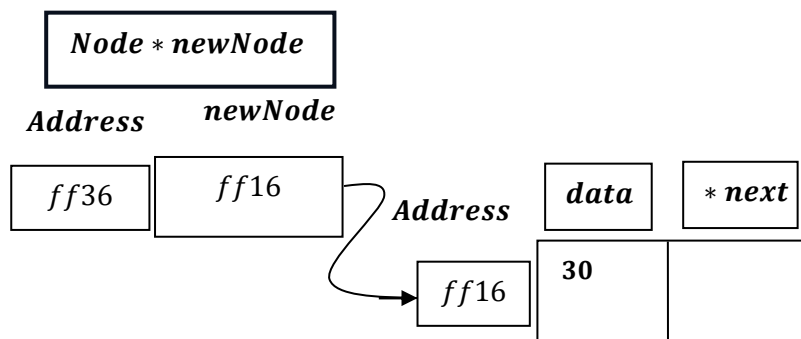
Similarly, for 2nd node we get:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

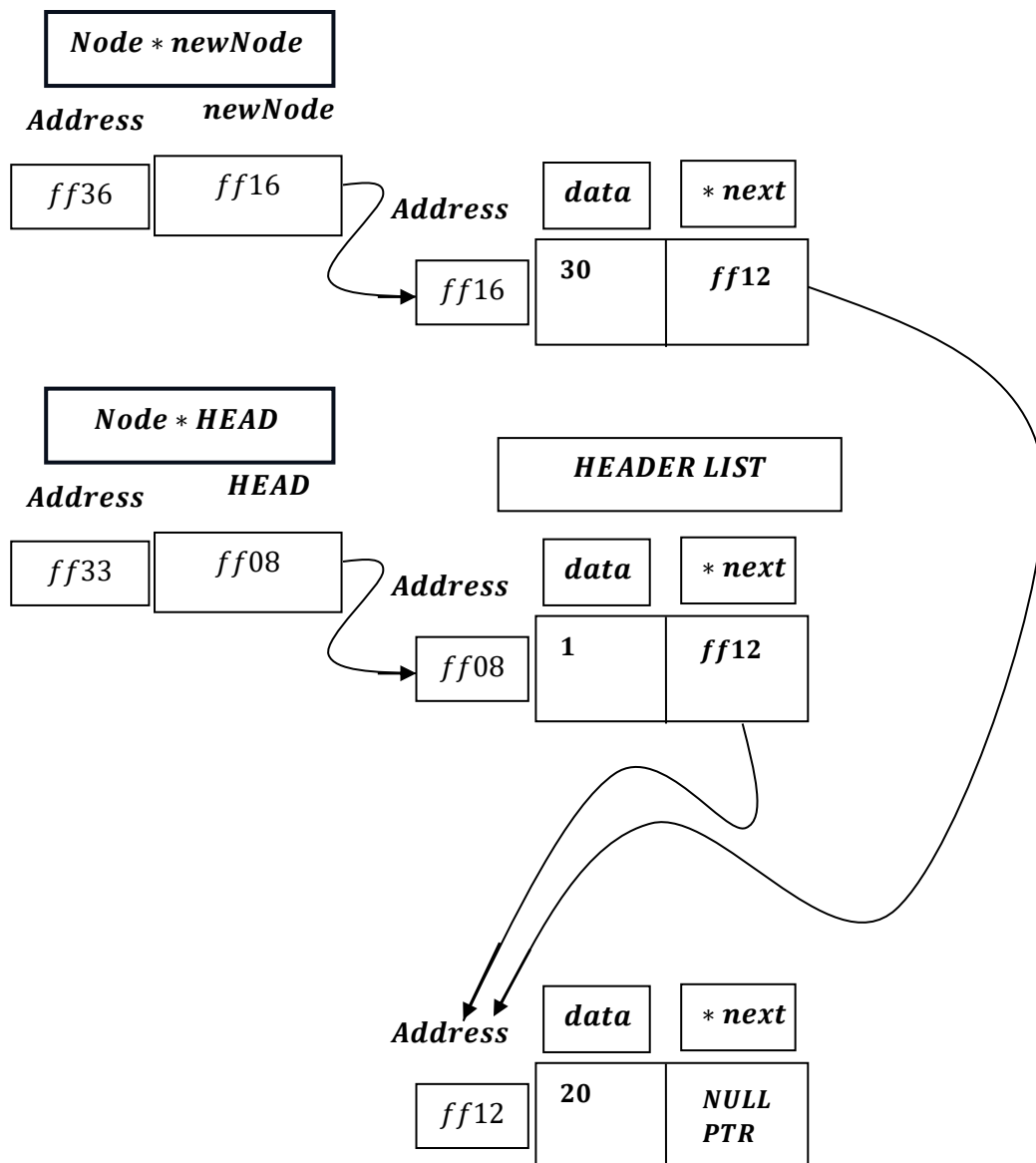


```
newNode->data = data;
```

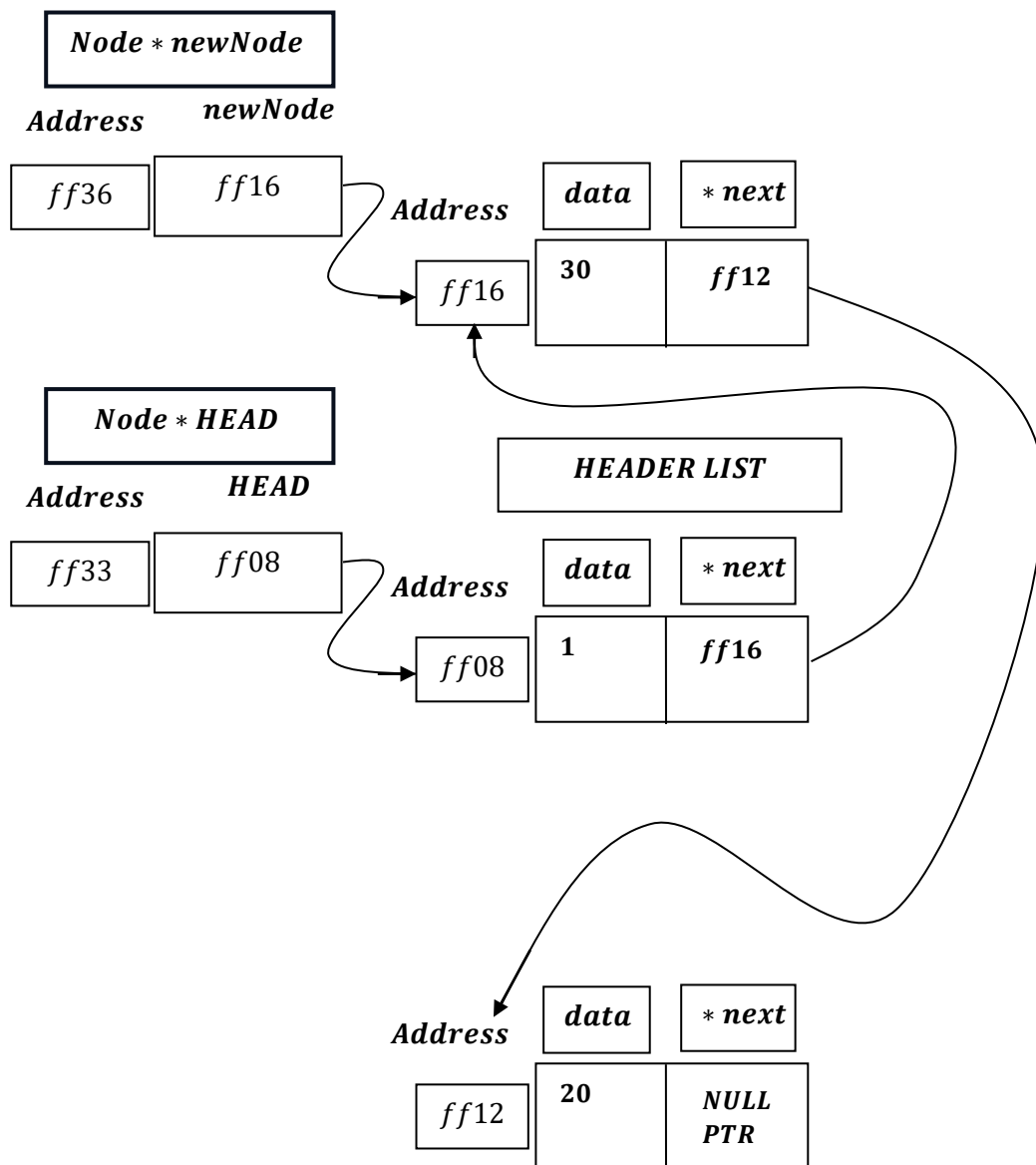
Lets say , $data = 30$.



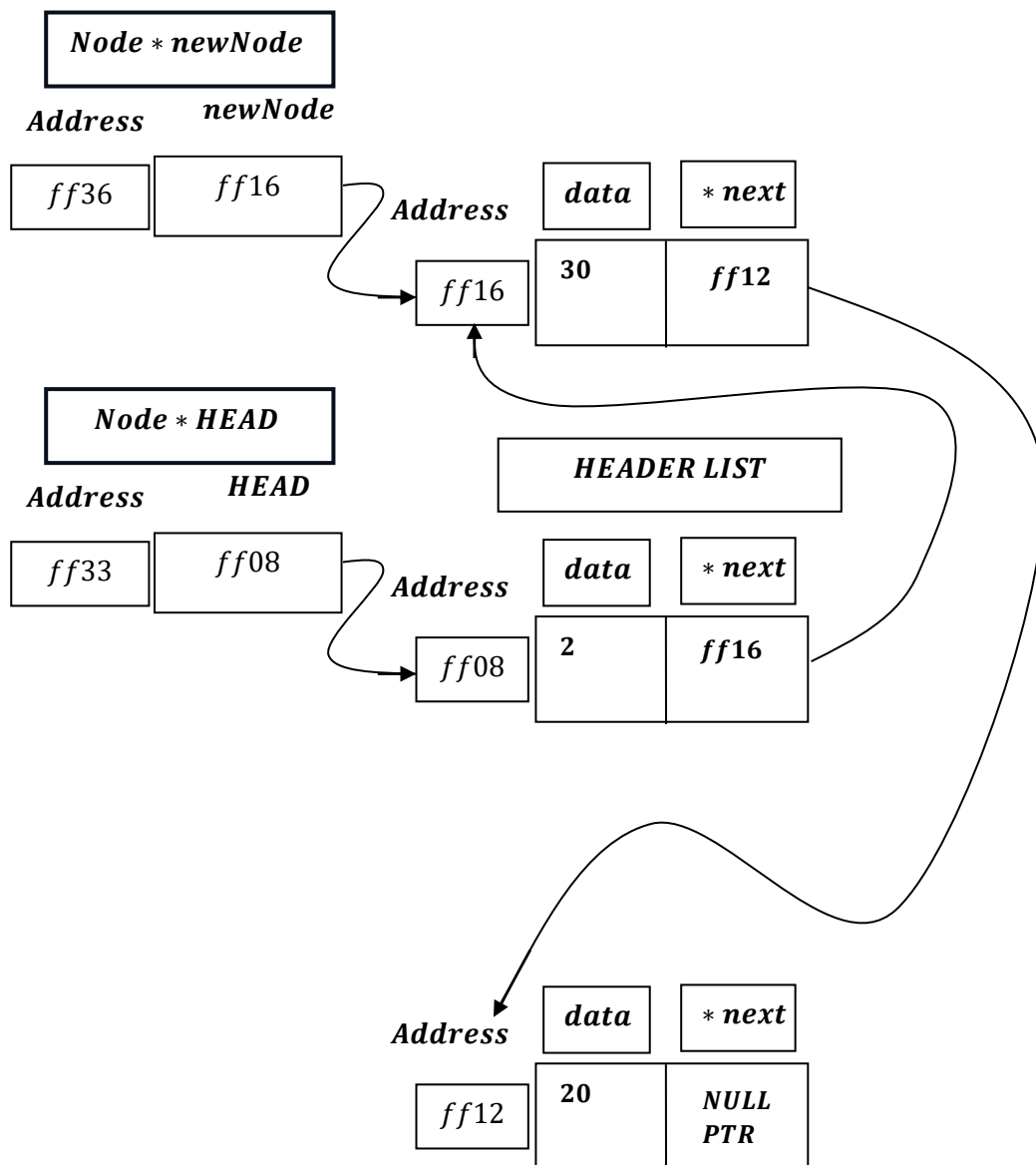
```
newNode->next = head->next;
```



```
head->next = newNode;
```



```
head->data++;
```



And we got our second list:

